

BEE 4750 Homework 1: Introduction to Using Julia

Name:

ID:

Due Date

Thursday, 9/11/25, 9:00pm

Overview

Instructions

- Problem 1 consist of a series of code snippets for you to interpret and debug. You will be asked to identify relevant error(s) and fix the code.
- Problem 2 asks you to write code to implement several simple functions using more general programming and some which are unique to Julia (which might mean that you need to look at the documentation or use Julia's help function, which is accessed with `?`, e.g. `?sum` for help with the `sum()` function) or to analyze Julia syntax.
- Problem 3 asks you to convert a verbal description of a wastewater treatment system into a Julia function, and then to use that function to explore the impact of different wastewater allocation strategies.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

```

Activating project at `~/Desktop/4750 Cloned Repositories/bee4750-hw1-wrn25-1`
  Updating registry at `~/.julia/registries/General.toml`
    Installed JpegTurbo_jll v3.1.3+0
    Installed SimpleTraits v0.9.5
    Installed Ghostscript_jll v9.55.1+0
    Installed ChainRulesCore v1.26.0
    Installed StaticArrays v1.9.15
    Installed IJulia v1.30.4
    Installed Interpolations v0.16.2
    Installed Graphs v1.13.1
    Installed Latexify v0.16.10
  Updating `~/Desktop/4750 Cloned Repositories/bee4750-hw1-wrn25-1/Project.toml`
[31c24e10] + Distributions v0.25.120
[bd48cda9] + GraphRecipes v0.5.14
[7073ff75] + IJulia v1.30.4
[b964fa9f] + LaTeXStrings v1.4.0
[91a5bcdd] + Plots v1.40.19
  Updating `~/Desktop/4750 Cloned Repositories/bee4750-hw1-wrn25-1/Manifest.toml`
[1520ce14] + AbstractTrees v0.4.5
[79e6a3ab] + Adapt v4.3.0
[66dad0bd] + AliasTables v1.1.3
[ec485272] + ArnoldiMethod v0.4.0
[13072b0f] + AxisAlgorithms v1.1.0
[d1d4a3ce] + BitFlags v0.1.9
[d360d2e6] + ChainRulesCore v1.26.0
[944b1d66] + CodecZlib v0.7.8
[35d6a980] + ColorSchemes v3.30.0
[3da002f7] + ColorTypes v0.11.5
[c3611d14] + ColorVectorSpace v0.10.0
[5ae59095] + Colors v0.13.1
[34da2185] + Compat v4.18.0
[f0e56b4a] + ConcurrentUtilities v2.5.0
[8f4d0f93] + Conda v1.10.2
[d38c429a] + Contour v0.6.3
[9a962f9c] + DataAPI v1.16.0
[864edb3b] + DataStructures v0.19.1
[8bb1440f] + DelimitedFiles v1.9.1
[31c24e10] + Distributions v0.25.120
[ffbed154] + DocStringExtensions v0.9.5
[460bff9d] + ExceptionUnwrapping v0.1.11
[411431e0] + Extents v0.1.6
[c87230d0] + FFMPEG v0.4.4
[1a297f60] + FillArrays v1.13.0

```

[53c48c17] + FixedPointNumbers v0.8.5
[1fa38f19] + Format v1.3.7
[28b8d3ca] + GR v0.73.17
[5c1252a2] + GeometryBasics v0.5.10
[4d00f742] + GeometryTypes v0.8.5
[bd48cda9] + GraphRecipes v0.5.14
[86223c79] + Graphs v1.13.1
[42e2da0e] + Grisu v1.0.2
[cd3eb016] + HTTP v1.10.17
[34004b35] + HypergeometricFunctions v0.3.28
[7073ff75] + IJulia v1.30.4
[d25df0c9] + Inflate v0.1.5
[a98d9a8b] + Interpolations v0.16.2
[92d709cd] + IrrationalConstants v0.2.4
[c8e1da08] + IterTools v1.10.0
[1019f520] + JLFzf v0.1.11
[692b3bcd] + JLLWrappers v1.7.1
[682c06a0] + JSON v0.21.4
[b964fa9f] + LaTeXStrings v1.4.0
[23fbe1c1] + Latexify v0.16.10
[2ab3a3ac] + LogExpFunctions v0.3.29
[e6f89c97] + LoggingExtras v1.1.0
[1914dd2f] + MacroTools v0.5.16
[739be429] + MbedTLS v1.1.9
[442fdcd] + Measures v0.3.2
[e1d29d7a] + Missings v1.2.0
[77ba4419] + NaNMath v1.1.3
[46757867] + NetworkLayout v0.4.10
[6fe1bfb0] + OffsetArrays v1.17.0
[4d8831e6] + OpenSSL v1.5.0
[bac558e1] + OrderedCollections v1.8.1
[90014a1f] + PDMats v0.11.35
[69de0a69] + Parsers v2.8.3
[ccf2f8ad] + PlotThemes v3.3.0
[995b91a9] + PlotUtils v1.4.3
[91a5bcd] + Plots v1.40.19
[aea7be01] + PrecompileTools v1.2.1
[21216c6a] + Preferences v1.5.0
[43287f4e] + PtrArrays v1.3.0
[1fd47b50] + QuadGK v2.11.2
[c84ed2f1] + Ratios v0.4.5
[3cdcf5f2] + RecipesBase v1.3.4
[01d81517] + RecipesPipeline v0.6.12

[189a3867] + Reexport v1.2.2
[05181044] + RelocatableFolders v1.0.1
[ae029012] + Requires v1.3.1
[79098fc4] + Rmath v0.8.0
[6c6a2e73] + Scratch v1.3.0
[992d4aef] + Showoff v1.0.3
[777ac1f9] + SimpleBufferStream v1.2.0
[699a6c99] + SimpleTraits v0.9.5
[a2af1166] + SortingAlgorithms v1.2.2
[276daf66] + SpecialFunctions v2.5.1
[860ef19b] + StableRNGs v1.0.3
[90137ffa] + StaticArrays v1.9.15
[1e83bf80] + StaticArraysCore v1.4.3
[10745b16] + Statistics v1.11.1
[82ae8749] + StatsAPI v1.7.1
[2913bbd2] + StatsBase v0.34.6
[4c63d2b9] + StatsFuns v1.5.0
[62fd8b95] + TensorCore v0.1.1
[3bb67fe8] + TranscodingStreams v0.11.3
[5c2747f8] + URIs v1.6.1
[1cfade01] + UnicodeFun v0.4.1
[1986cc42] + Unitful v1.24.0
[45397f5d] + UnitfulLatexify v1.7.0
[41fe7b60] + Unzip v0.2.0
[81def892] + VersionParsing v1.3.0
[efce3f68] + WoodburyMatrices v1.0.0
[c2297ded] + ZMQ v1.4.1
[6e34b625] + Bzip2_jll v1.0.9+0
[83423d85] + Cairo_jll v1.18.5+0
[ee1fde0b] + Dbus_jll v1.16.2+0
[5ae413db] + EarCut_jll v2.2.4+0
[2702e6a9] + EpollShim_jll v0.0.20230411+1
[2e619515] + Expat_jll v2.7.1+0
[b22a6f82] + FFMPEG_jll v7.1.1+0
[a3f928ae] + Fontconfig_jll v2.17.1+0
[d7e528f0] + FreeType2_jll v2.13.4+0
[559328eb] + FriBidi_jll v1.0.17+0
[0656b61e] + GLFW_jll v3.4.0+2
[d2c73de3] + GR_jll v0.73.17+0
[b0724c58] + GettextRuntime_jll v0.22.4+0
[61579ee1] + Ghostscript_jll v9.55.1+0
[7746bdde] + Glib_jll v2.84.3+0
[3b182d85] + Graphite2_jll v1.3.15+0

[2e76f6c2] + HarfBuzz_jll v8.5.1+0
 [aacddb02] + JpegTurbo_jll v3.1.3+0
 [c1c5ebd0] + LAME_jll v3.100.3+0
 [88015f11] + LERC_jll v4.0.1+0
 [1d63c593] + LLVMOpenMP_jll v18.1.8+0
 [dd4b983a] + LZ0_jll v2.10.3+0
 [e9f186c6] + Libffi_jll v3.4.7+0
 [7e76a0d4] + Libglvnd_jll v1.7.1+1
 [94ce4f54] + Libiconv_jll v1.18.0+0
 [4b2f31a3] + Libmount_jll v2.41.1+0
 [89763e89] + Libtiff_jll v4.7.1+0
 [38a345b3] + Libuuid_jll v2.41.1+0
 [e7412a2a] + Ogg_jll v1.3.6+0
 [458c3c95] + OpenSSL_jll v3.5.2+0
 [efe28fd5] + OpenSpecFun_jll v0.5.6+0
 [91d4177d] + Opus_jll v1.5.2+0
 [36c8627f] + Pango_jll v1.56.3+0
 [30392449] + Pixman_jll v0.44.2+0
 [c0090381] + Qt6Base_jll v6.8.2+1
 [629bc702] + Qt6Declarative_jll v6.8.2+1
 [ce943373] + Qt6ShaderTools_jll v6.8.2+1
 [e99dba38] + Qt6Wayland_jll v6.8.2+1
 [f50d1b31] + Rmath_jll v0.5.1+0
 [a44049a8] + Vulkan_Loader_jll v1.3.243+0
 [a2964d1f] + Wayland_jll v1.24.0+0
 [ffd25f8a] + XZ_jll v5.8.1+0
 [f67eecfb] + Xorg_libICE_jll v1.1.2+0
 [c834827a] + Xorg_libSM_jll v1.2.6+0
 [4f6342f7] + Xorg_libX11_jll v1.8.12+0
 [0c0b7dd1] + Xorg_libXau_jll v1.0.13+0
 [935fb764] + Xorg_libXcursor_jll v1.2.4+0
 [a3789734] + Xorg_libXdmcp_jll v1.1.6+0
 [1082639a] + Xorg_libXext_jll v1.3.7+0
 [d091e8ba] + Xorg_libXfixes_jll v6.0.1+0
 [a51aa0fd] + Xorg_libXi_jll v1.8.3+0
 [d1454406] + Xorg_libXinerama_jll v1.1.6+0
 [ec84b674] + Xorg_libXrandr_jll v1.5.5+0
 [ea2f1a96] + Xorg_libXrender_jll v0.9.12+0
 [c7cfdc94] + Xorg_libxcb_jll v1.17.1+0
 [cc61e674] + Xorg_libxkbfile_jll v1.1.3+0
 [e920d4aa] + Xorg_xcb_util_cursor_jll v0.1.5+0
 [12413925] + Xorg_xcb_util_image_jll v0.4.1+0
 [2def613f] + Xorg_xcb_util_jll v0.4.1+0

[975044d2] + Xorg_xcb_util_keysyms_jll v0.4.1+0
[0d47668e] + Xorg_xcb_util_renderutil_jll v0.3.10+0
[c22f9ab0] + Xorg_xcb_util_wm_jll v0.4.2+0
[35661453] + Xorg_xkbcomp_jll v1.4.7+0
[33bec58e] + Xorg_xkeyboard_config_jll v2.44.0+0
[c5fb5394] + Xorg_xtrans_jll v1.6.0+0
[8f1865be] + ZeroMQ_jll v4.3.6+0
[3161d3a3] + Zstd_jll v1.5.7+1
[35ca27e7] + eudev_jll v3.2.14+0
[214eeab7] + fzf_jll v0.61.1+0
[a4ae2306] + libaom_jll v3.12.1+0
[0ac62f75] + libass_jll v0.17.4+0
[1183f4f0] + libdecor_jll v0.2.2+0
[2db6ffa8] + libevdev_jll v1.13.4+0
[f638f0a6] + libfdk_aac_jll v2.0.4+0
[36db933b] + libinput_jll v1.28.1+0
[b53b4c65] + libpng_jll v1.6.50+0
[a9144af2] + libsodium_jll v1.0.21+0
[f27f6e37] + libvorbis_jll v1.3.8+0
[009596ad] + mtdev_jll v1.1.7+0
[1270edf5] + x264_jll v10164.0.1+0
[dfaa095f] + x265_jll v4.1.0+0
[d8fb68d0] + xkbcommon_jll v1.9.2+0
[0dad84c5] + ArgTools v1.1.2
[56f22d72] + Artifacts v1.11.0
[2a0f44e3] + Base64 v1.11.0
[ade2ca70] + Dates v1.11.0
[8ba89e20] + Distributed v1.11.0
[f43a241f] + Downloads v1.6.0
[7b1f6079] + FileWatching v1.11.0
[b77e0a4c] + InteractiveUtils v1.11.0
[b27032c2] + LibCURL v0.6.4
[76f85450] + LibGit2 v1.11.0
[8f399da3] + Libdl v1.11.0
[37e2e46d] + LinearAlgebra v1.11.0
[56ddb016] + Logging v1.11.0
[d6f4376e] + Markdown v1.11.0
[a63ad114] + Mmap v1.11.0
[ca575930] + NetworkOptions v1.2.0
[44cfe95a] + Pkg v1.11.0
[de0858da] + Printf v1.11.0
[3fa0cd96] + REPL v1.11.0
[9a3f8284] + Random v1.11.0

```

[ea8e919c] + SHA v0.7.0
[9e88b42a] + Serialization v1.11.0
[1a1011a3] + SharedArrays v1.11.0
[6462fe0b] + Sockets v1.11.0
[2f01184e] + SparseArrays v1.11.0
[f489334b] + StyledStrings v1.11.0
[4607b0f0] + SuiteSparse
[fa267f1f] + TOML v1.0.3
[a4e569a6] + Tar v1.10.0
[8dfed614] + Test v1.11.0
[cf7118a7] + UUIDs v1.11.0
[4ec0a83e] + Unicode v1.11.0
[e66e0078] + CompilerSupportLibraries_jll v1.1.1+0
[deac9b47] + LibCURL_jll v8.6.0+0
[e37daf67] + LibGit2_jll v1.7.2+0
[29816b5a] + LibSSH2_jll v1.11.0+1
[c8ffd9c3] + MbedTLS_jll v2.28.6+0
[14a3606d] + MozillaCACerts_jll v2023.12.12
[4536629a] + OpenBLAS_jll v0.3.27+1
[05823500] + OpenLibm_jll v0.8.5+0
[efcefd7] + PCRE2_jll v10.42.0+1
[bea87d4a] + SuiteSparse_jll v7.7.0+0
[83775a58] + Zlib_jll v1.2.13+1
[8e850b90] + libblastrampoline_jll v5.11.0+0
[8e850ede] + nghttp2_jll v1.59.0+0
[3f19e933] + p7zip_jll v17.4.0+2

```

Info Packages marked with and have new versions available. Those with may be upg

Building IJulia → `~/julia/scratchspaces/44cfe95a-1eb2-52ea-b672-e2afdf69b78f/87567b508

Precompiling project...

```

374.2 ms  EpollShim_jll
377.6 ms  Xorg_libXau_jll
385.3 ms  Xorg_libICE_jll
404.5 ms  Rmath_jll
525.8 ms  SimpleTraits
713.9 ms  QuadGK
386.4 ms  mtdev_jll
383.6 ms  Xorg_libXdmcp_jll
424.8 ms  EarCut_jll
846.2 ms  ChainRulesCore
483.1 ms  JpegTurbo_jll
425.0 ms  LZ0_jll
389.2 ms  Xorg_xtrans_jll
369.7 ms  libevdev_jll

```

375.1 ms	eudev_jll
429.6 ms	OpenSpecFun_jll
354.5 ms	DBus_jll
384.6 ms	Wayland_jll
366.7 ms	Xorg_xcb_util_jll
742.6 ms	Rmath
615.0 ms	ChainRulesCore → ChainRulesCoreSparseArraysExt
1440.7 ms	OpenSSL
831.5 ms	LogExpFunctions → LogExpFunctionsChainRulesCoreExt
353.8 ms	Xorg_libX11_jll
927.1 ms	Libtiff_jll
966.1 ms	Ghostscript_jll
385.9 ms	Xorg_libXfixes_jll
389.3 ms	Xorg_libxkbfile_jll
376.6 ms	Xorg_libXinerama_jll
2813.2 ms	ColorSchemes
381.8 ms	Xorg_libXrandr_jll
352.0 ms	Xorg_libXcursor_jll
1778.4 ms	SpecialFunctions
765.0 ms	Pango_jll
376.4 ms	Xorg_libXi_jll
409.1 ms	Xorg_xkbcomp_jll
4752.6 ms	StaticArrays
852.2 ms	HypergeometricFunctions
1270.9 ms	SpecialFunctions → SpecialFunctionsChainRulesCoreExt
583.0 ms	ColorVectorSpace → SpecialFunctionsExt
2550.8 ms	Latexify
606.3 ms	Xorg_xkeyboard_config_jll
608.8 ms	StaticArrays → StaticArraysChainRulesCoreExt
614.1 ms	StaticArrays → StaticArraysStatisticsExt
653.8 ms	Adapt → AdaptStaticArraysExt
1315.0 ms	ArnoldiMethod
833.3 ms	Latexify → SparseArraysExt
1371.3 ms	UnitfulLatexify
1478.8 ms	StatsFuns
389.6 ms	libdecor_jll
7578.1 ms	IJulia
738.5 ms	Qt6ShaderTools_jll
1505.3 ms	GeometryTypes
1397.8 ms	StatsFuns → StatsFunsChainRulesCoreExt
2140.4 ms	Interpolations
7183.9 ms	PlotUtils
1176.9 ms	GR_jll


```

1684.3 ms    Qt6Declarative_jll
1079.4 ms    Interpolations → InterpolationsUnitfulExt
4039.9 ms    Graphs
 348.4 ms    Qt6Wayland_jll
2114.2 ms    PlotThemes
4547.7 ms    Distributions
11931.1 ms   HTTP
2750.1 ms    RecipesPipeline
8251.6 ms    GeometryBasics
 934.1 ms    Distributions → DistributionsChainRulesCoreExt
1000.6 ms    Distributions → DistributionsTestExt
1091.2 ms    NetworkLayout
2276.8 ms    GR
1189.5 ms    NetworkLayout → NetworkLayoutGraphsExt
1936.4 ms    GraphRecipes
32859.9 ms   Plots
2836.6 ms    Plots → IJuliaExt
2904.8 ms    Plots → GeometryBasicsExt
2993.2 ms    Plots → UnitfulExt

```

76 dependencies successfully precompiled in 53 seconds. 164 already precompiled.

Standard Julia practice is to load all needed packages at the top of a file. If you need to load any additional packages in any assignments beyond those which are loaded by default, feel free to add a `using` statement, though [you may need to install the package](#).

```

using Random
using Plots
using GraphRecipes
using LaTeXStrings
using Distributions

```

```

# this sets a random seed, which ensures reproducibility of random number generation. You sh
Random.seed!(1)

```

```
TaskLocalRNG()
```

Problems (Total: 30 Points)

Problem 1 (12 points)

The following subproblems all involve code snippets that require debugging. You are encouraged to use online resources (*e.g.* Julia documentation and forums, Stack Overflow, Reddit,

etc) to help find diagnose error messages and find solutions, but make sure that you clearly document which resources you used and how you used them.

Problem 1.1

You've been tasked with writing code to identify the minimum value in an array. You cannot use a predefined function. Your colleague suggested the function below, but it does not return the minimum value.

```
function minimum(array)
    # initialize the minimum value counter
    min_value = array[1]
    # update minimum values
    for i in 1:length(array)
        if array[i] < min_value
            min_value = array[i]
        end
    end
    # return found minimum
    return min_value
end

array_values = [89, 90, 95, 100, 100, 78, 99, 98, 100, 95]
@show minimum(array_values);
```

```
minimum(array_values) = 78
```

What is the logical flaw in the code? Fix it and show that your fixed code solves the problem.

-> I changed the line “min_value = 0” to “min_value = array[1]”, the first entry in the given array. Since all the entries in the example array_values were above 0, the code would never return anything besides 0 unless we had a negative value in the array. The function now returns the correct minimum number.

Problem 1.2

Your team is trying to compute the average grade for your class, but the following code produces an `UndefVarError`.

```
# enter student grade vector
student_grades = [89, 90, 95, 100, 100, 78, 99, 98, 100, 95]
# compute class average
function class_average(grades)
    average_grade = mean(grades)
    return average_grade
end

@show class_average(student_grades);
```

```
class_average(student_grades) = 94.4
```

What does this `UndefVarError` mean? Why does this code produce one? Fix the error and solve the problem.

→ This problem gives an `UndefVarError` because the `@show` line at the end asked for “`average_grade`” which is a variable that is only defined within the function “`class_average`”. To fix this problem I changed the last line to “`@show class_average(student_grades)`” so we actually utilize the written function. Also edited the function slightly, changing the variable inside the `mean()` function to “`grades`”, so that the function outputs the specific array inputted into `class_average()`, rather than just the average of “`student_grades`”, the predefined array which may not be what we want to calculate the average of.

Problem 1.3

Your team wants to know the expected payout of an old Italian dice game called *passadieci* (which was analyzed by Galileo as one of the first examples of a rigorous study of probability). The goal of *passadieci* is to get at least an 11 from rolling three fair, six-sided dice. Your strategy is to compute the average wins from 1,000 trials, but the code you’ve written below produces a `MethodError`.

```
# function to simulate a passadieci roll: 3 6-sided dice
function passadieci()
    # this rand() call samples 3 values from the vector [1, 6]
    roll = rand(1:6, 3)
    return roll
end
# set number of trials and initialize outcome vector
n_trials = 1000
counter = 0
# simulate number of passadieci rolls and count wins
for i = 1:n_trials
```

```

    if sum(passadieci()) >= 11
        counter = counter + 1
    end
end
win_prob = counter / n_trials # compute average number of wins
@show win_prob;

```

win_prob = 0.483

What does this `MethodError` mean? Why does this code produce one? Fix the error and solve the problem.

→ This code gives a `MethodError` for several reasons, but primarily because boolean values were being assigned to array entries of an integer variable from the “outcomes[i] = (sum(passadieci()) > 11)” line. This error shows up when we call functions with none of the correct defined methods. To solve this problem I used a different process of counting the number of wins, by using an if statement and a counter variable to add one to the counter everytime the sum of the random dice rolls was greater than or equal to 11. Then dividing this number of wins by 1,000 gives the expected probability of success.

Problem 1.4

You’re interested in writing some code to remove the mean of a vector from all of its components. You’ve written the following code and tried to test it on a random vector, but your code returns a `MethodError`.

```

# function to remove mean from a vector
function remove_mean(vect)
    # function to compute the mean
    function compute_mean(vect)
        element_sum = 0 # initialize sum
        # compute mean and return
        for v in vect
            element_sum += v
        end
        return element_sum / length(vect)
    end
end

m = compute_mean(vect) # compute mean
# return demeaned vector
new_vect = vect.- m

```

```

    return new_vect;
end
random_vect = rand(5);
@show random_vect;
new_vect = remove_mean(random_vect);
@show new_vect;

```

```

random_vect = [0.1649421746173687, 0.9042378964373602, 0.21663396098069898, 0.81304121543108, 0.2315043732786];
new_vect = [-0.41659466753502383, 0.32270105428496765, -0.36490288117169356, 0.2315043732786, -0.41659466753502383];

```

What does this `MethodError` mean? Why does this code produce one? Fix the error and solve the problem.

→ `MethodError` in this case refers to the line “return vect - m”. vect is a vector, m is a float, and the subtraction function does not have a defined method dimension match between these two variable types. To solve this problem I used the “.-” function to subtract the mean equally from all vector entries, and I added a new vector called new_vect to take on the new updated values to simplify the code. I decreased the number of elements in the original array from 1_000 to 5 to simplify the mean calculations so I could double check them. But the code works with any number of random array elements.

Problem 2 (18 points)

Cheap Plastic Products, Inc. is operating a plant that produces $100\text{m}^3/\text{day}$ of wastewater that is discharged into Pristine Brook. The wastewater contains $1\text{kg}/\text{m}^3$ of YUK, a toxic substance. The US Environmental Protection Agency has imposed an effluent standard on the plant prohibiting discharge of more than $20\text{kg}/\text{day}$ of YUK into Pristine Brook.

Cheap Plastic Products has analyzed two methods for reducing its discharges of YUK. Method 1 is land disposal, which costs $X_1^2/20$ dollars per day, where X_1 is the amount of wastewater disposed of on the land (m^3/day). With this method, 20% of the YUK applied to the land will eventually drain into the stream (*i.e.*, 80% of the YUK is removed by the soil).

Method 2 is a chemical treatment procedure which costs $\$1.50$ per m^3 of wastewater treated. The chemical treatment has an efficiency of $e = 1 - 0.005X_2$, where X_2 is the quantity of wastewater (m^3/day) treated. For example, if $X_2 = 50\text{m}^3/\text{day}$, then $e = 1 - 0.005(50) = 0.75$, so that 75% of the YUK is removed.

Cheap Plastic Products is wondering how to allocate their wastewater between these three disposal and treatment methods (land disposal, chemical treatment, and direct disposal) to meet the effluent standard while keeping costs manageable.

The flow of wastewater through this treatment system is shown in Figure 1. Modify the edge labels (by editing the `edge_labels` dictionary in the code producing Figure 1) to show how the wastewater allocations result in the final YUK discharge into Pristine Brook. For the `edge_label` dictionary, the tuple (i, j) corresponds to the arrow going from node i to node j . The syntax for any entry is $(i, j) \Rightarrow \text{"label text"}$, and the label text can include mathematical notation if the string is prefaced with an L, as in `L"x_1"` will produce x_1 .

```
A = [0 1 1 1;
      0 0 0 1;
      0 0 0 1;
      0 0 0 0]

names = ["Plant", "Land Treatment", "Chem Treatment", "Pristine Brook"]
# modify this dictionary to add labels
edge_labels = Dict{Tuple{Int, Int}, String}((1, 2) => "", (1, 3) => "", (1, 4) => "", (2, 4) => "", (3, 4) => "")
shapes = [:hexagon, :rect, :rect, :hexagon]
xpos = [0, -1.5, -0.25, 1]
ypos = [1, 0, 0, -1]

p = graphplot(A, names=names, edgelabel=edge_labels, markersize=0.15, markershapes=shapes, margin=0.5)
display(p)
```

Problem 2.1

Formulate a mathematical model for the treatment cost and the amount of YUK that will be discharged into Pristine Brook based on the wastewater allocations. This is best done with some equations and supporting text explaining the derivation. Make sure you include, as additional equations in the model, any needed constraints on relevant values. You can find some basics on writing mathematical equations using the LaTeX typesetting syntax [here](#), and a cheatsheet with LaTeX commands can be found on the course website's [Resources page](#).

Problem 2.2

Implement your systems model as a Julia function which computes the resulting YUK discharge and cost for a particular treatment plan. You can return multiple values from a function with a [tuple](#), as in:

```
function multiple_return_values(x, y)
    return (x+y, x*y)
end

a, b = multiple_return_values(2, 5)
```

```
@show a;  
@show b;
```

```
a = 7  
b = 10
```

To evaluate the function over vectors of inputs, you can *broadcast* the function by adding a decimal `.` before the function arguments and accessing the resulting values by writing a *comprehension* to loop over the individual outputs in the vector:

```
x = [1, 2, 3, 4, 5]  
y = [6, 7, 8, 9, 10]  
  
output = multiple_return_values.(x, y)  
a = [out[1] for out in output]  
b = [out[2] for out in output]  
@show a;  
@show b;
```

```
a = [7, 9, 11, 13, 15]  
b = [6, 14, 24, 36, 50]
```

Make sure you comment your code appropriately to make it clear what is going on and why.

Problem 2.3

Use your function to experiment with 1,000 different combinations of wastewater discharge and treatment. You can do this with either a grid search or by sampling from a [Dirichlet distribution](#) (a $\text{Dirichlet}(1, n)$ distribution will generate uniformly-weighted n -dimensional vectors whose components add up to 1; see the [Distributions.jl documentation](#) for how to sample from probability distributions in Julia). Plot the results of these experiments. Do any satisfy the YUK effluent standard (plot this as well as a dashed red line). What was the cost of solutions satisfying the standard? What can you say about the tradeoff between treatment cost and YUK discharge? You don't have to find an "optimal" solution to this problem, but what do you think would be needed to find a better solution?

Problem 2.4

Find the strategies which minimize cost and YUK discharge (these will be different strategies) analytically and find the values of the objective metrics. Plot these values in the plot that you created for Problem 2.3. How do their values compare to the spread of values that you found in that problem? Would you select either of them (explain why or why not)?

References

List any external references consulted, including classmates.